WEPL

Pre-Production Engineering Audit

Fintech-Social Platform · Community Finance · ROSCA · M-Pesa

Report Date: May 22, 2026

Classification: CONFIDENTIAL — Engineering Leadership Only

Executive Summary

This report presents the findings of a comprehensive pre-production audit of the WEPL platform — a Django-based fintech-social application handling community savings, ROSCA cycles, welfare funds, emergency advances, and M-Pesa payment flows. The audit was conducted against 15 engineering dimensions covering security, financial integrity, concurrency, scalability, observability, and code quality.

The platform demonstrates strong foundational thinking: a clean state-machine ledger, idempotency-key design, optimistic row-locking, and domain-event decoupling are all present. However, several CRITICAL defects make the platform **NOT safe for production deployment** as-of this audit. Three issues in particular would result in either complete financial-engine failure or silent loss of customer payments from day one.

Audit Scores by Area

Area	Score / 10	Verdict
Architecture	6	Sound skeleton, dual-path liability
Security	4	PIN throttle missing; webhook unsigned
Financial Integrity	3	Ledger divergence; non-idempotent keys
API Design	5	IDOR holes; missing pagination
Database	6	Good locking; missing indexes; CASCADE risk
Django-Specific	6	Clean settings; minor anti-patterns
Real-Time / WebSockets	5	Auth gap; no rate limits; thread risk
Celery / Async	3	Financial queue dead; retry logic broken
Scalability	5	Will hit walls at ~5K users
Observability / DevOps	5	Sentry wired; metrics absent
Testing	2	Tests pass wrong scenarios silently
Failure Scenarios	3	Silent data loss paths present
Code Quality	6	Generally readable; critical bugs hidden
Performance	5	N+1 queries; unindexed paths
Data Consistency	5	Dual-write divergence possible

Overall Readiness Score: 4.3 / 10 — NOT PRODUCTION READY

Critical Blockers — Must Fix Before Launch

The following issues are deployment blockers. Any one of them alone would cause material financial harm, complete service failure, or irrecoverable data loss in production.

CB-1	Financial Celery queue never consumed
Location	`docker-compose.yml worker command omits -Q financial`
Impact	ALL B2C disbursements, standing orders, and reconciliation tasks queue forever — zero payouts ever execute; group funds accumulate but nobody gets paid

CB-2	B2C retry logic permanently disabled
Location	`apps/ledger/tasks.py — _handle_payout_failure() called on first exception, before retry`
Impact	Any transient M-Pesa network hiccup permanently fails and reverses the payout; users receive error notifications for temporary API timeouts

CB-3	africastalking missing from requirements.txt
Location	`requirements.txt — package not listed`
Impact	Fresh deployment crashes at startup; phone authentication completely broken

CB-4	STK callback swallows all exceptions
Location	`apps/mpesa/views.py STKCallbackView._on_success() bare except`
Impact	Customer pays via M-Pesa STK Push; money leaves their account; exception in processing silently discards payment; no credit, no retry

CB-5	Welfare & advance repayment idempotency keys use wall clock
Location	`apps/contributions/services.py WelfareService.contribute(), AdvanceService.repay()`
Impact	Celery task retry or network duplicate creates a second LedgerEntry; fund balance double-counted; advance marked repaid twice

CB-6	PaymentService.create_payment() never writes to LedgerEntry
Location	`apps/payments/services.py — ContributionTransaction created, LedgerEntry omitted`
Impact	Parallel payment path (manual reconciliation) permanently diverges from ledger; balance queries return wrong totals; audit trail incomplete

CB-7	ContributionJoinRequest.__str__ AttributeError
Location	`apps/contributions/models.py — duplicate __str__ references self.voter, self.amendment_id`
Impact	Any admin page load, Django shell inspection, or log line that stringifies a join request raises AttributeError; admin panel broken for contribution onboarding

CB-8	M-Pesa webhooks have no signature or IP validation
Location	`apps/mpesa/views.py B2CResultView, STKCallbackView — AllowAny, no validation`
Impact	Attacker posts fake B2C success callback; platform credits a payout that never happened; direct financial fraud vector

1. Architecture Review

1.1 Overall Structure

WEPL is a Django monolith with a logical app-per-domain layout: users, communities, contributions, ledger, payments, mpesa, notifications, conversations, activity. The separation of the immutable ledger from mutable domain state is architecturally sound. Domain events decouple the service layer from side-effect delivery. State machines with optimistic concurrency on financial models are the right pattern.

1.2 Dual Code Path Problem

The most dangerous architectural defect is the existence of two independent paths for recording a payment: **ContributionService.contribute()** (the primary, ledger-aware path) and **PaymentService.create_payment()** (the manual reconciliation path). The second path writes a ContributionTransaction but never touches LedgerEntry. Any payment recorded via the payments app will permanently diverge the mutable balance from the immutable ledger.

[CRITICAL] Dual payment paths cause permanent ledger divergence

Location:	apps/payments/services.py:PaymentService.create_payment() — no LedgerEntry write
Root Cause:	PaymentService was added without integrating the ledger writer introduced later. The ContributionTransaction model predates LedgerEntry and was never migrated away.
Impact:	All manually-reconciled payments produce wrong balance totals. Audit trail incomplete. Reconciliation task in apps/contributions/tasks.py will flag these as phantom discrepancies.
Fix:	Add write_ledger_entry() call inside PaymentService.create_payment() after the ContributionTransaction.objects.create() call. Use the same idempotency_key pattern as ContributionService: f'payment-{payment.id}-{paying_user.id}'
Ideal Redesign:	Eliminate PaymentService entirely. Route all manual reconciliation through ContributionService.contribute() with a source='manual' flag.

1.3 Asgi / Channels Configuration

config/asgi.py hardcodes DJANGO_SETTINGS_MODULE to production at import time. Any test runner or staging deployment that relies on environment-variable overrides will silently use production settings for the ASGI stack only, while Django's WSGI path picks up the override correctly. This creates a split-config surface that is almost impossible to debug under load.

2. Security Audit

[CRITICAL] **M-Pesa webhooks accept requests from any IP with no signature check**

Location:	apps/mpesa/views.py — B2CResultView, STKCallbackView: permission_classes = [AllowAny], no HMAC
Root Cause:	Safaricom provides a security credential for signing B2C payloads. No validation was implemented.
Impact:	Attacker constructs a valid-looking B2C success payload and POSTs it. Platform credits a disbursement that Safaricom never made. Direct financial fraud, no detection.
Fix:	1. Restrict to Safaricom CIDR ranges at nginx/firewall level. 2. Validate the SecurityCredential signature on every B2C callback. 3. Add request logging with IP for all webhook endpoints.
Ideal Redesign:	Implement a WebhookSignaturePermission class that validates HMAC-SHA256 of the raw request body against the Safaricom public key. Reject any request failing validation with HTTP 401 and log the attempt to Sentry.

[HIGH] **PIN login endpoint has no rate-limit throttle applied**

Location:	apps/users/views.py:PINLoginView — no throttle_classes defined; config/settings/base.py defines 'pin_login': '5/min' but it is never referenced
Root Cause:	The throttle scope is defined in REST_FRAMEWORK settings but no view applies throttle_classes = [ScopedRateThrottle] with throttle_scope = 'pin_login'.
Impact:	An attacker can brute-force the PIN at full Django request throughput (~300 req/s). The Redis lockout after 5 attempts is the only protection — bypassed by distributing attempts across a 30-minute window at just under the lockout threshold.
Fix:	Add to PINLoginView: throttle_classes = [ScopedRateThrottle] throttle_scope = 'pin_login'
Ideal Redesign:	Replace Redis lockout with a token-bucket algorithm per (phone_number, IP). Add progressive delays: 1s, 2s, 4s... after each failure. Alert on >10 failures/hour.

[HIGH] **JWT token transmitted in WebSocket URL query parameter**

Location:	apps/conversations/jwt_middleware.py — extracts token from ?token=
Root Cause:	WebSocket upgrade URLs appear in nginx access logs, browser history, referrer headers, and CDN logs in plaintext.
Impact:	Any log aggregation system (ELK, Datadog, Splunk) will capture live JWT tokens. Token exfiltration from logs grants full session access for up to 60 minutes.
Fix:	Pass the token in the WebSocket subprotocol header (Sec-WebSocket-Protocol: access_token,) or send it as the first message after connection and authenticate before allowing any channel operations.
Ideal Redesign:	Implement a short-lived WebSocket ticket: POST /ws/ticket/ returns a 30-second single-use UUID stored in Redis. WebSocket connects with ?ticket=UUID. Middleware exchanges ticket for user identity.

[HIGH] **WebSocket membership not re-validated on each message**

Location:	apps/conversations/consumers.py:ConversationConsumer.receive() — no auth check
Root Cause:	Membership is validated only at connect(). A user removed from a community mid-session retains WebSocket access until their connection drops.
Impact:	Removed members can read and send messages for the duration of their connection. In a long-lived mobile app this could be hours.
Fix:	In receive(), call an async DB check: await sync_to_async(CommunityMembership.objects.filter)(community=..., user=self.user, is_active=True).aexists() and close the connection if False.
Ideal Redesign:	Cache membership state in the channel layer group. Emit a 'member_removed' group message when a user is removed; the consumer closes its own connection on receipt.

[MEDIUM] **development.py CORS/ALLOWED_HOSTS wildcards risk staging bleed**

Location:	config/settings/development.py — ALLOWED_HOSTS=['*'], CORS_ALLOW_ALL_ORIGINS=True
Root Cause:	No guard prevents development settings from being used in a CI/staging environment.
Impact:	If a staging deployment accidentally loads development settings (wrong env var), all origins can make credentialed cross-origin requests to the API.
Fix:	Add an explicit assertion: assert not os.environ.get('PRODUCTION') at the top of development.py. Add CORS_ALLOWED_ORIGIN_REGEXES for localhost patterns only.
Ideal Redesign:	Use django-environ's Env.bool() with required=True on DJANGO_DEBUG and raise ImproperlyConfigured if the environment is ambiguous.

3. Financial Integrity Audit

[CRITICAL] **Welfare and advance repayment idempotency keys include wall-clock timestamp**

Location: apps/contributions/services.py — WelfareService.contribute() and AdvanceService.repay()

Root Cause: The idempotency key uses timezone.now().strftime('%Y%m%d%H%M%S'). If the Celery task retries within the same second, the key is identical and is correctly deduplicated. But if the retry fires even one second later, a new key is generated and a second LedgerEntry is created.

Impact: Double-counting of welfare contributions and advance repayments. Fund balances inflated. Members appear to have repaid advances they haven't. Irrecoverable without manual ledger correction.

Fix: Derive idempotency keys from stable, business-meaningful identifiers only: welfare_contrib: f'welfare-contrib-{fund_id}-{user.id}-{period}' advance_repay: f'advance-repay-{advance_id}-{sequence_number}'

Ideal Redesign: Create a PaymentIntent model with a client-generated idempotency UUID. All service calls accept this UUID. The UUID is the idempotency key. No timestamps anywhere in financial key derivation.

[CRITICAL] **B2C task marks transaction FAILED on first attempt, disabling all retries**

Location: apps/ledger/tasks.py:execute_b2c_payout() — _handle_payout_failure() called in except block before self.retry()

Root Cause: The failure handler transitions FinancialTransaction to FAILED state, which is a terminal state. The retry logic at the bottom of the function checks if state == FAILED and returns early. The Celery retry therefore never fires.

Impact: Every M-Pesa B2C call that gets a network timeout, rate limit, or 5xx is immediately reversed. Users receive 'Payment failed' for what would be a successful retry. Effective M-Pesa reliability drops to single-shot success rate (~85%).

Fix: Replace _handle_payout_failure() in the except block with raise self.retry(exc=exc, countdown=60). Only call _handle_payout_failure() in the on_failure Celery hook, which fires only after all retries are exhausted.

Ideal Redesign: Add a RETRYING state to FinancialTransaction. Transition to RETRYING on retry, FAILED only on final failure. Add retry_count and last_error fields for observability.

[HIGH] **WelfareFund.balance not rolled back when B2C disbursement fails**

Location: apps/mpesa/views.py:_on_b2c_failure() — no balance restoration logic

Root Cause: B2C success callback calls WelfareService._mark_disbursed() which decrements WelfareFund.balance. But the B2C failure callback has no matching increment.

Impact: If M-Pesa sends a failure result for a disbursement, the fund balance stays decremented despite no actual outflow. Repeat disbursement attempts are blocked by 'insufficient balance' errors even though the money is still in the fund.

Fix: In _on_b2c_failure(), restore the fund balance: WelfareFund.objects.filter(id=...).update(balance=F('balance') + amount). Write a REVERSAL LedgerEntry.

Ideal Redesign: Make the entire disbursement lifecycle atomic via a saga pattern: each step registers a compensating transaction. Failure at any step triggers the compensation chain.

[HIGH] **STK callback exception handler silently discards customer payments**

Location:	apps/mpesa/views.py:STKCallbackView._on_success() — bare except: pass
Root Cause:	The bare except clause around the payment processing block catches all exceptions including DatabaseError, IntegrityError, and any service layer exception.
Impact:	Customer money leaves their M-Pesa wallet. Exception occurs (e.g. DB connection blip). Payment is never credited. No retry scheduled. No alert fired. Customer calls support. Manual reconciliation required for every incident.
Fix:	Replace bare except with: except Exception as exc: logger.error(...) sentry_sdk.capture_exception(exc) self.retry(exc=exc) Never swallow exceptions in financial callbacks.
Ideal Redesign:	Move STK callback processing to a Celery task. The webhook view does one thing: claim the callback row with UPDATE WHERE status='PENDING', enqueue the task, return 200. All financial logic runs in the task with full retry and dead-letter handling.

4. API Design Audit

[HIGH] **IDOR: Any authenticated user can read any community's contributions**

Location:	apps/contributions/views.py:CommunityContributionsView.get() — no membership check
Root Cause:	The view filters contributions by community_id from the URL. No check verifies that request.user is a member of that community.
Impact:	Any logged-in user can enumerate all contribution types, amounts, member balances, and financial history for any private community by iterating community IDs.
Fix:	Add at the start of get(): if not CommunityMembership.objects.filter(community_id=community_id, user=request.user, is_active=True).exists(): raise PermissionDenied
Ideal Redesign:	Extract a membership_required() decorator or a get_community_or_403() shortcut. Apply it as a mixin to every view that accepts community_id.

[HIGH] **Three endpoints return unbounded querysets with no pagination**

Location:	apps/communities/views.py:DiscoverCommunitiesView, apps/contributions/views.py:OpenContributionsView, apps/contributions/views.py:WelfareActivityView (in-memory sort)
Root Cause:	No pagination class applied; Django ORM returns the full table.
Impact:	At 1,000 communities or 10,000 contribution events, these endpoints return multi-megabyte JSON responses, exhaust database memory, and time out under load.
Fix:	Apply CursorPagination or PageNumberPagination to all list endpoints. Add pagination_class to each ViewSet. Add DB-level ORDER BY before slicing.
Ideal Redesign:	Enforce a global default: REST_FRAMEWORK = {'DEFAULT_PAGINATION_CLASS': ..., 'PAGE_SIZE': 20}. Override per-view where needed. Any view without explicit pagination_class = None should inherit the default.

[MEDIUM] **PIN transmitted in X-Pin header — logged by most HTTP middleware**

Location:	apps/contributions/views.py:ContributeView — pin = request.META.get('HTTP_X_PIN')
Root Cause:	Custom request headers appear in Django debug toolbar, DRF browsable API logs, nginx access logs with verbose header logging, and APM traces.
Impact:	Raw PIN values visible in log aggregation systems. If logs are breached, attackers obtain PINs in plaintext.
Fix:	Move PIN to the request body as a dedicated field. Ensure the field is excluded from all serializer __repr__ and logging. Mark it sensitive in APM (Sentry, Datadog).
Ideal Redesign:	Implement a challenge-response: server issues a nonce, client sends HMAC(PIN, nonce). Server verifies. PIN never transmitted in plaintext.

5. Database Audit

[HIGH] **CASCADE delete on User propagates to ALL financial data**

Location: apps/communities/models.py:Community.created_by FK — on_delete=CASCADE, apps/contributions/models.py:Contribution.created_by FK — on_delete=CASCADE

Root Cause: Django default on_delete propagation was not overridden for creator foreign keys.

Impact: Deleting or deactivating any user who created a community or contribution cascades deletion to ALL associated financial records: contributions, ledger entries, disbursements, welfare claims. Irreversible data loss.

Fix: Change all creator FKs to on_delete=PROTECT or on_delete=SET_NULL with null=True. Never allow User deletion; instead implement soft-delete (is_active=False).

Ideal Redesign: Add a pre_delete signal on User that raises PermissionDenied if the user has any financial records. Enforce soft-delete-only at the model level.

[MEDIUM] **Notification model foreign keys are plain IntegerFields — no FK constraints**

Location: apps/notifications/models.py — community_id, contribution_id, join_request_id, conversation_id are IntegerField, not ForeignKey

Root Cause: Likely chosen to avoid cross-app import cycles at model definition time.

Impact: Orphaned notification rows when referenced objects are deleted. No ORM traversal, no Django admin inline editing. Stale notification pointers cause 404s in the notification detail endpoint.

Fix: Use GenericForeignKey (ContentType framework) or define explicit ForeignKey with on_delete=SET_NULL, null=True, db_constraint=False to avoid circular imports.

Ideal Redesign: Create a NotificationTarget polymorphic model. Each notification references one target via ContentType. Enables ORM traversal, cascade cleanup, and type safety.

[MEDIUM] **Missing database indexes on high-frequency query paths**

Location: Multiple models — see detail

Root Cause: Indexes not explicitly defined on frequently-filtered non-PK columns.

Impact: At scale, unindexed queries become sequential scans. Specific hot paths:
LedgerEntry(contribution_id, entry_date) — financial history queries
FinancialTransaction(idempotency_key) — unique constraint implies index but verify
Notification(user_id, is_read, created_at) — notification feed
ContributionTransaction(contribution_id, user_id) — balance calculations
StandingOrder(next_run_at, is_active) — scheduled task query

Fix: Add db_index=True or explicit Meta.indexes with Index(fields=[...]) on each. Run EXPLAIN ANALYZE on the five queries above under load to validate.

Ideal Redesign: Instrument slow query logging in PostgreSQL (log_min_duration_statement=100ms). Review pg_stat_user_indexes weekly for zero-usage indexes.

6. Celery & Async Task Audit

[CRITICAL] **Financial Celery queue missing from worker command**

Location: docker-compose.yml:celery service command: celery -A config worker -l info -Q default,notifications,payments --concurrency=4 (missing: financial)

Root Cause: The task routes in settings/base.py send apps.ledger.tasks.* and apps.contributions.tasks.* to the 'financial' queue. The worker never subscribes to it.

Impact: ALL B2C payouts, ROSCA payout scheduling, standing order execution, balance reconciliation, and stale transaction recovery tasks queue indefinitely. No user ever receives a disbursement. Standing orders never fire. The Redis queue grows unbounded.

Fix: Add financial to the worker command: -Q default,notifications,payments,financial Consider running the financial queue on a dedicated worker with --concurrency=2 to prevent financial task starvation during notification spikes.

Ideal Redesign: Use separate Celery worker services per queue in docker-compose. Add a health check that alerts if any queue depth exceeds 100 tasks for >5 minutes.

[HIGH] **reconcile_balances task iterates ALL records in Python**

Location: apps/contributions/tasks.py:reconcile_balances() — Contribution.objects.filter(is_active=True) full scan, Python loop

Root Cause: No database-level aggregation; all rows fetched into memory, balanced checked in Python.

Impact: At 10,000 active contributions, this task fetches millions of LedgerEntry rows into the Celery worker process. Memory exhaustion, OOM kill, or task timeout. The reconciliation that's supposed to catch bugs becomes the bug.

Fix: Rewrite as a single SQL aggregation: SELECT c.id, c.current_amount, SUM(CASE WHEN le.direction='CREDIT' THEN le.amount ELSE -le.amount END) FROM contributions JOIN ledger_entries GROUP BY c.id HAVING c.current_amount != SUM(...)

Ideal Redesign: Run reconciliation as a PostgreSQL stored procedure or a Django RawSQL query. Paginate using keyset pagination on contribution.id. Process in batches of 500.

[MEDIUM] **Notification send_notification task not idempotent**

Location: apps/notifications/tasks.py:send_notification — max_retries=3, no idempotency key

Root Cause: Celery retry creates a new Notification row each time. No deduplication check.

Impact: A transient DB connection error causes 3 duplicate notifications delivered to the user for every payment event. Trust erosion; notification spam.

Fix: Use Notification.objects.get_or_create(idempotency_key=...) where the key is derived from the triggering event (e.g. f'payment-{payment_id}-{notification_type}').

Ideal Redesign: Adopt a transactional outbox pattern: write notification intent rows in the same transaction as the domain event. A separate poller delivers them exactly-once.

7. Real-Time / WebSocket Audit

[HIGH] **No message size limit on WebSocket receive()**

Location:	apps/conversations/consumers.py:ConversationConsumer.receive() — no size check
Root Cause:	Django Channels passes the full message body to receive() without built-in size limits.
Impact:	Any connected user sends a 10 MB message. Consumer allocates 10 MB RAM per worker thread. With 4 Daphne workers × 100 connections each, a coordinated burst exhausts server RAM.
Fix:	Add at the top of receive(): if len(text_data) > 4096: # 4KB limit await self.send(json.dumps({'error': 'Message too large'})) return
Ideal Redesign:	Configure CHANNEL_LAYERS with a max_length constraint. Add nginx client_max_body_size for WebSocket upgrade connections.

[MEDIUM] **Typing indicator has no rate limiting — Redis saturation vector**

Location:	apps/conversations/consumers.py:_handle_typing() — unconditional channel_layer.group_send()
Root Cause:	Every keystroke event triggers a Redis PUBLISH. No debounce, no per-user throttle.
Impact:	A malicious client sends 1,000 typing events/second. Redis PUBLISH throughput saturates, degrading ALL channel layer operations including message delivery. Denial of service against the real-time system.
Fix:	Track last_typing_sent per user in the consumer instance. Only publish if > 2 seconds since last event: now = time.time() if now - self._last_typing > 2.0: self._last_typing = now await self.channel_layer.group_send(...)
Ideal Redesign:	Rate limit at the Channels middleware level. Apply the same ScopedRateThrottle concept to WebSocket event types using a Redis token bucket per user.

[MEDIUM] **Multiple sync_to_async DB calls per message hit Django thread pool**

Location:	apps/conversations/consumers.py — sync_to_async wrapping ORM calls in receive()
Root Cause:	Each sync_to_async call runs in Django's threadpool executor (default: cpu_count × 5). Under high WebSocket concurrency, all threads are occupied with DB calls.
Impact:	Thread pool exhaustion blocks all sync_to_async calls across all consumers, freezing every connected WebSocket client simultaneously.
Fix:	Migrate ORM calls to async-native Django ORM syntax (Django 4.1+): message = await Message.objects.acreate(...) members = [m async for m in community.members.aiterator()]
Ideal Redesign:	Adopt full async Django views and consumers. Eliminate sync_to_async entirely for the hot path. Keep sync_to_async only for third-party libraries without async support.

8. Testing Audit

The test suite has a structural defect that makes it more dangerous than no tests: several tests assert on the wrong exception types, meaning they silently PASS even when the system is completely broken. This creates false confidence.

[CRITICAL] Tests assert wrong exception types — silently pass on broken code

Location:	apps/contributions/tests.py — multiple test cases
Root Cause:	Services raise django.core.exceptions.PermissionDenied but tests catch the built-in PermissionError. These are different classes with no inheritance relationship.
Impact:	assertRaises(PermissionError) passes vacuously because PermissionDenied is not a subclass of PermissionError. The permission check could be completely removed from the service and the test would still pass. Authorization is effectively untested.
Fix:	Replace all assertRaises(PermissionError) with assertRaises(PermissionDenied) (from django.core.exceptions). Add a linting rule: grep for 'PermissionError' in test files and fail CI.
Ideal Redesign:	Write a base TestCase mixin that automatically validates the exception type matches the service contract. Use mypy strict mode to catch exception type mismatches.

[CRITICAL] Three test methods call non-existent service methods

Location:	apps/contributions/tests.py: WelfareTests.test_get_or_create_fund calls WelfareService.get_or_create_fund() (actual: get_or_create_community_fund()) WelfareTests.test_majority_vote_disburses_claim calls WelfareService.vote_on_claim() (method does not exist) EmergencyAdvanceTests.test_admin_can_approve asserts status == 'APPROVED' (actual state after approve_advance is DISBURSED)
Root Cause:	Tests were written against an earlier API design and never updated after refactoring. They raise AttributeError immediately and are excluded from the suite by the test runner as erroring (not failing), masking the coverage gap.
Impact:	These tests appear to cover critical welfare and emergency flows but actually provide zero coverage. Production bugs in these flows will not be caught by CI.
Fix:	Fix immediately: 1. get_or_create_fund → get_or_create_community_fund 2. vote_on_claim → implement the missing method or rewrite the test 3. assert status == 'DISBURSED' (not 'APPROVED')
Ideal Redesign:	Add pytest-strict-markers and configure CI to treat any test error as a failure. Add mypy type checking to service method calls in tests.

[HIGH] **No integration tests for M-Pesa callback flows**

Location:	tests/ directory — no mpesa tests found
Root Cause:	M-Pesa integration is complex and failure-prone. No tests cover STK callback processing, B2C result handling, or reconciliation logic.
Impact:	Silent financial data loss (CB-4) was never caught because no test exercises the callback path with a simulated exception.
Fix:	Write integration tests using responses or httpretty to mock the M-Pesa API. Test: successful STK callback credits balance; failed callback does not; duplicate callback is idempotent; malformed payload returns 400.
Ideal Redesign:	Set up a Safaricom sandbox environment for staging. Run full E2E payment flows in CI against the sandbox before every production deploy.

9. Performance & Scalability

[HIGH] **N+1 queries in ContributionSerializer**

Location: apps/contributions/serializers.py: get_participant_count() — COUNT(*) per contribution get_user_balance() — LedgerEntry aggregate per contribution

Root Cause: SerializerMethodField calls fire individual DB queries per object in the list response.

Impact: A list of 50 contributions fires 100+ queries. Latency grows linearly. At 200 contributions visible on the home feed, a single page load issues 400 queries.

Fix: Use prefetch_related + annotate: qs.annotate(participant_count=Count('participants'), user_balance=Subquery(...)) Pass the annotation values into the serializer context.

Ideal Redesign: Define a ContributionSummarySerializer for list views with only denormalized fields. Use ContributionDetailSerializer only for single-object endpoints.

[HIGH] **M-Pesa access token fetched on every API call**

Location: apps/mpesa/services.py:MpesaService._get_access_token() — HTTP call every invocation

Root Cause: No caching layer; Daraja access tokens are valid for 3600 seconds.

Impact: Each STK Push or B2C initiation makes two HTTP calls to Safaricom: one for the access token, one for the actual API. Doubles latency per payment, hits Daraja rate limits under load.

Fix: Cache the token in Redis: token = cache.get('mpesa_access_token') if not token: token = self._fetch_access_token() cache.set('mpesa_access_token', token, timeout=3500)

Ideal Redesign: Wrap _get_access_token() with a thread-safe singleton refresh pattern. Add a Celery Beat task that pre-fetches the token every 58 minutes.

[MEDIUM] **MyCommunitiesView uses 4 LEFT JOINs for activity sorting**

Location: apps/communities/views.py:MyCommunitiesView — complex GREATEST annotation query

Root Cause: The view annotates each community with the latest activity timestamp across 4 different related models using GREATEST(). This generates a complex SQL query with 4 LEFT JOINs and 4 subqueries per community.

Impact: For a user in 20 communities, this is a join-heavy query that cannot use standard indexes. Execution time grows super-linearly with community size and activity volume.

Fix: Add a last_activity_at field to Community, updated via a signal or service method whenever activity occurs. Sorting becomes a simple ORDER BY index scan.

Ideal Redesign: Implement a Redis sorted set per user: community_activity:{user_id} storing community_id → timestamp scores. Update on write. Read for feed ordering with O(log N).

Scaling Failure Points

The following are the predicted load thresholds at which specific subsystems will fail without architectural changes:

User Load	Subsystem	Failure Mode	Primary Cause
~500 users	Notification queue	Backlog grows unbounded	Duplicate sends from non-idempotent retries
~1,000 users	DiscoverCommunities endpoint	Response timeout / OOM	Full table scan, no pagination

~2,000 users	MyCommunitiesView	P99 latency > 5s	4-JOIN annotation query with no covering index
~5,000 users	reconcile_balances task	OOM kill in Celery worker	Full Python iteration over all ledger entries
~10,000 users	WebSocket typing indicators	Redis saturation	Unbounded PUBLISH rate per connected user
~50,000 users	Celery broker (shared Redis)	Redis memory exhausted	Broker and cache on same Redis instance

10. Observability & DevOps

What's Present (Good)

- Sentry SDK integrated for Django and Celery with DSN from environment
- recover_stale_processing_transactions() Celery Beat task logs CRITICAL alerts for stuck transactions
- Celery task routing to named queues enables queue-level monitoring
- CONN_MAX_AGE=60 for persistent DB connections in production
- Non-root Docker user (appuser) for container security

Critical Gaps

[HIGH] **No Prometheus / StatsD metrics**

Location: Observability gap

Root Cause: No business metrics: payment success rate, OTP delivery rate, disbursement lag, active WebSocket connections. Impossible to build SLOs or detect degradation trends without page-level alerts.

Impact: No business metrics: payment success rate, OTP delivery rate, disbursement lag, active WebSocket connections. Impossible to build SLOs or detect degradation trends without page-level alerts.

Fix:

[HIGH] **No dead-letter queue for failed Celery tasks**

Location: Observability gap

Root Cause: Tasks that exhaust retries are silently dropped. Failed B2C payouts (after the retry bug is fixed) disappear with no operator notification. Add a dead-letter queue and PagerDuty alert for any DLQ entry.

Impact: Tasks that exhaust retries are silently dropped. Failed B2C payouts (after the retry bug is fixed) disappear with no operator notification. Add a dead-letter queue and PagerDuty alert for any DLQ entry.

Fix:

[HIGH] **No structured logging**

Location: Observability gap

Root Cause: logger.info/error calls use string formatting. Cannot query logs by payment_id, community_id, or user_id in ELK/Splunk. Add structlog or python-json-logger with standard fields on every log line.

Impact: logger.info/error calls use string formatting. Cannot query logs by payment_id, community_id, or user_id in ELK/Splunk. Add structlog or python-json-logger with standard fields on every log line.

Fix:

[MEDIUM] **migrate on every container start is dangerous**

Location: Observability gap

Root Cause: docker-compose.yml runs python manage.py migrate as the web service entrypoint. In a multi-replica deployment, two containers racing to apply the same migration causes IntegrityError or table-lock timeout. Run migrations as a one-shot init container, not as the web process entrypoint.

Impact: docker-compose.yml runs python manage.py migrate as the web service entrypoint. In a multi-replica deployment, two containers racing to apply the same migration causes IntegrityError or table-lock timeout. Run migrations as a one-shot init container, not as the web process entrypoint.

Fix:

[MEDIUM] **No health check endpoint**

Location: Observability gap

Root Cause: docker-compose.yml has no healthcheck for the web service. A crashed Django process is not detected until a user reports an error. Add GET /health/ that checks DB connectivity, Redis ping, and returns 200/503.

Impact: docker-compose.yml has no healthcheck for the web service. A crashed Django process is not detected until a user reports an error. Add GET /health/ that checks DB connectivity, Redis ping, and returns 200/503.

Fix:

11. Top Risks, Breach Vectors & Failure Modes

Financial Fraud Vectors

- POST forged B2C callback to /mpesa/b2c/result/ → platform credits fake disbursement (no sig validation)
- Submit duplicate STK callback within 1-second window → dual credit if idempotency key collides
- Brute-force PIN via distributed attempt below lockout threshold → account takeover
- Exploit IDOR on CommunityContributionsView to enumerate all community finances
- Inject arbitrary amount into STK Push body (float vs Decimal rounding exploitable)
- Replay expired JWT with modified stage claim (if token blacklist has gap)

Financial Corruption Scenarios

- Welfare contribution Celery retry fires after >1s → double LedgerEntry, inflated fund balance
- PaymentService.create_payment() used for manual reconciliation → LedgerEntry never written → permanent divergence between contribution.current_amount and ledger sum
- B2C disbursement fails → WelfareFund.balance decremented but never restored → 'insufficient balance' blocks legitimate future disbursements
- CASCADE delete of User cascades to Contribution, LedgerEntry, DisbursementRequest → entire community's financial history deleted
- User deactivated mid-standing-order cycle → standing order fires but contribution check fails → ROSCA rotation skips participant silently
- AmendmentService._apply() sets fields directly without state machine protection → concurrent amendment and contribution creates inconsistent state

Infrastructure Failure Modes

- Redis goes down → OTP cache cleared → all in-flight authentication fails; PIN lockout state cleared → lockouts lifted; Celery broker down → all tasks queue locally and are lost on restart
- Database connection pool exhausted (no PgBouncer) → 503 cascade under moderate load spike
- Celery financial worker OOM on reconcile_balances → task silently fails; balance discrepancies accumulate undetected
- M-Pesa Daraja API outage → STK Push fails; B2C retries accumulate; after outage resolves, all retries fire simultaneously and may exceed rate limits
- Shared Redis for broker + cache → large Celery job queue evicts cache keys → OTP lookups return None → all authentications fail

12. Architectural Evolution Roadmap

Phase 1: Fix Blockers (Week 1)

- Add financial to Celery worker queue list (docker-compose.yml)
- Fix B2C retry logic — move _handle_payout_failure() to on_failure hook
- Add africastalking to requirements.txt
- Fix STK callback bare except — capture exception, log to Sentry, retry task
- Fix welfare and advance repayment idempotency keys (remove wall clock)
- Add LedgerEntry write to PaymentService.create_payment()
- Fix ContributionJoinRequest duplicate __str__ method
- Fix tests — correct exception types, method names, expected states
- Add M-Pesa IP allowlist + request logging at nginx level

Phase 2: Security Hardening (Week 2-3)

- Apply ScopedRateThrottle with pin_login scope to PINLoginView
- Move PIN to request body; exclude from all logging
- Implement WebSocket ticket exchange (replace ?token= query param)
- Add HMAC signature validation to all M-Pesa webhook views
- Fix IDOR — add membership check to CommunityContributionsView and ContributionPaymentsView
- Change User FK on_delete to PROTECT or SET_NULL everywhere
- Fix Notification model IntegerFields → ForeignKey(null=True, on_delete=SET_NULL)

Phase 3: Reliability & Observability (Week 3-5)

- Separate Redis broker from Redis cache (two Redis instances)
- Add Prometheus metrics: payment success rate, disbursement lag, queue depth
- Implement dead-letter queue for financial tasks + PagerDuty alert
- Add structured logging with structlog (payment_id, community_id as fields)
- Move migrate out of container entrypoint into init container
- Add GET /health/ endpoint with DB + Redis connectivity check
- Cache M-Pesa access token in Redis (3500s TTL)
- Add re-validation of membership in WebSocket receive()

Phase 4: Performance (Week 5-8)

- Add pagination to DiscoverCommunities, OpenContributions, WelfareActivity
- Fix N+1 in ContributionSerializer with annotate() and prefetch_related()
- Rewrite reconcile_balances as a SQL aggregation query with keyset pagination
- Add last_activity_at field to Community; replace 4-JOIN annotation
- Add message size limit to WebSocket consumer

- Add typing indicator debounce (2s minimum interval)
- Migrate WebSocket consumer ORM calls to Django async ORM (acreate, afilter)
- Add DB indexes: LedgerEntry(contribution_id, entry_date), Notification(user_id, is_read)

Phase 5: Scale Preparation (Month 2-3)

- Introduce PgBouncer for connection pooling
- Implement transactional outbox pattern for notification delivery
- Add Redis sorted set for community activity feed ordering
- Separate Celery worker processes per queue (financial workers isolated)
- Consider read replicas for analytical queries (reconciliation, reporting)
- Implement M-Pesa webhook signature validation with Safaricom public key
- Add end-to-end tests against M-Pesa sandbox in CI

Appendix: Additional Issues

[MEDIUM] **PINService.set_pin() double-saves the user**

Location: apps/users/services.py:PINService.set_pin()

Root Cause: Calls user.set_pin(pin) which calls user.save(), then calls user.is_pin_set=True; user.save() again.

Impact: Two database writes per PIN set operation. Second write overwrites the first with a slightly stale user state if any concurrent modification occurred between the two saves.

Fix: Remove the redundant is_pin_set = True; user.save() from PINService.set_pin(). The User.set_pin() method already sets is_pin_set = True and saves.

[MEDIUM] **asgi.py hardcodes production settings module**

Location: config/asgi.py — os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.production')

Root Cause: ASGI entrypoint unconditionally sets the settings module to production. Any test that imports the ASGI application picks up production settings.

Impact: WebSocket tests in CI connect to production Redis and production DB if env vars are set. Or they fail silently if production env vars are absent.

Fix: Use DJANGO_SETTINGS_MODULE from the environment, do not setdefault to production. Let the deployment orchestrator set the correct module.

[LOW] **AmendmentService._apply() bypasses state machine**

Location: apps/contributions/services.py:AmendmentService._apply()

Root Cause: Directly sets contribution fields without going through Contribution.transition_to(). No optimistic locking; no state validation.

Impact: A concurrent amendment and contribution could produce an inconsistent state. The amendment silently overwrites a contribution that was updated between the amendment vote and its application.

Fix: Wrap _apply() in select_for_update() on the Contribution row. Validate that the contribution is in an ACTIVE state before applying.

[LOW] **M-Pesa STKPushView uses float() for amount**

Location: apps/mpesa/views.py:STKPushView — amount = float(request.data.get('amount'))

Root Cause: IEEE 754 floating-point representation causes rounding for values like 100.10.

Impact: M-Pesa amount sent as 100.09999999... Safaricom may reject or round unpredictably. Financial amounts must never pass through float.

Fix: Use Decimal: amount = Decimal(str(request.data.get('amount'))). Validate with a Serializer field: DecimalField(max_digits=10, decimal_places=2).

[LOW] **M-Pesa C2B reconciliation uses ambiguous 9-digit phone suffix matching**

Location: apps/mpesa/services.py:reconcile_c2b() — phone_number__endswith=phone[-9:]

Root Cause: Phone numbers in Africa are not globally unique on their last 9 digits.

Impact: Two users with phone numbers differing only in country code match the same suffix. Payment credited to wrong account.

Fix: Normalize all phone numbers to E.164 format at storage time (+254XXXXXXXXX). Match on the full normalized number. Use a phone normalization library (phonenumbers).

WEPL Pre-Production Engineering Audit

Generated May 22, 2026 · CONFIDENTIAL · Engineering Leadership Only

This document contains a complete audit of the WEPL codebase as reviewed. All findings reflect the state of the code at time of review. Re-audit recommended after each phase of remediation.